

Guía: preparar los assets 3D en Blender

Pie, pierna y zapatos para el dataset sintético — AR Shoe Try-On

Proyecto: ar-shoes (3DCommerceCL) Versión: 1.2 — julio 2026 Fuente: training/GUIA_ESCENA_BLENDER.md

Lo primero: NO tenés que armar ninguna escena

Lo primero: NO tenés que armar ninguna escena. Luces, cámara, piso, HDRIs, máscaras y randomización los genera solo `0b_blender_render.py` en cada render. Lo único que hay que producir en Blender son **los archivos GLB** que ese script consume. Esta guía es el contrato exacto de esos archivos (consolidando el brief al experto 3D + lo que los scripts esperan).

Qué archivos hay que producir

Archivo	Contenido	Estado actual
<code>training/models/foot.glb</code>	UN pie (derecho) + variantes de pierna + 6 Empties de keypoints	❌ placeholder (cono rosado) — es lo que hay que reemplazar
<code>training/models/shoes/*.glb</code>	≥10 zapatos variados, cada uno ya alineado al pie	❌ solo hay 1 mocasín
<code>training/hdri/*.hdr</code>	10-20 HDRIs de iluminación real	❌ vacío (descarga, no Blender)
<code>training/floor_textures/*.jpg</code>	10-20 fotos de pisos reales	❌ vacío (descarga, no Blender)

1. El pie con pierna: `foot.glb`

Reglas de escala y orientación (regla dura del proyecto)

- **1 unidad de Blender = 1 metro.** El pie adulto debe medir $\approx 0.26\text{ m}$ de largo (verificalo en el panel N → Item → Dimensions).
- El pie apunta hacia **+Y** (los dedos hacia Y positivo), arriba es **+Z**.
- El **origen del objeto en el talón, apoyado en el suelo** (el punto del talón queda en X=0, Y=0, Z=0).
- Es **UN pie derecho** solamente. El izquierdo NO se modela: el entrenamiento lo genera espejando (por eso los keypoints usan nombres anatómicos, ver abajo).
- Antes de exportar: seleccionar todo → `Ctrl+A` → **All Transforms** (aplicar transformaciones). Los scripts NUNCA reescalan ni reposicionan los GLB — lo que exportás es lo que se renderiza.

De dónde sacar un pie realista (sin esculpirlo)

Opción recomendada si no vas a esculpir: **MakeHuman** (gratis, mallas resultantes CC0 — sin problema de licencia para dataset comercial): 1. Generá un cuerpo (variá tono de piel entre exports si querés diversidad). 2. Importalo en Blender (MPFB o export .obj), entrá en Edit Mode y **borrá todo desde media pantorrilla**

hacia arriba (**B** box-select + **X**). 3. Cerrá el hueco del corte: seleccioná el borde abierto → **F** (fill) o **Alt+F** . 4. Ya viene con textura de piel. Ajustá escala hasta que el pie mida 0.26 m.

Alternativas: BlenderKit (filtrar licencia CC0), o el asset de Sketchfab actual **solo si su licencia permite uso comercial/ML** (pendiente de verificar — T0.4 del ROADMAP).

Las variantes de pierna (crítico para el domain gap)

En el uso real siempre se ve pierna o pantalón sobre el zapato — sin esto el modelo aprende mal la frontera pierna/zapato. Hay que incluir **3 variantes**, y el script activa UNA al azar por render:

Nombre EXACTO del objeto	Qué es
<code>leg_bare</code>	La pierna con piel desde el tobillo hacia arriba
<code>leg_pants_dark</code>	Un bajo de pantalón oscuro cayendo sobre el tobillo (tubo de tela alcanza)
<code>leg_pants_light</code>	Ídem en tela clara (jean claro / beige)

¿Hasta dónde llega la pierna: media pantorrilla o la rodilla?

Recomendado: ~30 cm sobre el tobillo (media pantorrilla / debajo de la rodilla). No hace falta llegar a la rodilla y tampoco conviene quedarse corto:

- **Muy corta (< 15 cm):** el "tubo" se ve entero dentro del cuadro con su tapa superior flotando — el modelo aprende un objeto que no existe en la realidad (en una foto real la pierna siempre sale del cuadro o llega hasta la ropa).
- **~30 cm (recomendado):** en casi todos los ángulos la pierna **sale del cuadro** por arriba, que es exactamente lo que pasa en una foto real. Es el rango que mejor transfiere.
- **Hasta la rodilla (45 cm+):** no aporta más (la parte de arriba casi nunca entra en cuadro) y obliga a la cámara a alejarse, achicando el pie en el encuadre. No está mal, pero no suma.

Nota técnica (ya resuelta en el script): el script calcula la distancia de cámara con el tamaño del *pie*, así que con una pierna larga la cámara cenital quedaba **por debajo del tope de la pierna** y el render salía arruinado. Se corrigió: ahora la cámara nunca entra en la esfera que contiene toda la escena (pierna incluida) y el near-clip bajó a 1 cm. Con cualquier largo razonable funciona — pero 30 cm sigue siendo el mejor encuadre.

Que la pierna sea cónica (más fina en el tobillo, más ancha arriba: ~7 cm de diámetro en el tobillo, ~11 cm arriba). Un cilindro recto y grueso desde el tobillo tapa el pie en las tomas desde atrás. Y **cerrá la tapa de arriba** (que no sea un tubo hueco).

⚠ **Importante — por NOMBRE DE OBJETO, no por colección:** el formato GLB no conserva las colecciones de Blender. El script detecta las variantes por el **nombre del objeto** (prefijo `leg_`, insensible a mayúsculas; los sufijos `.001` no molestan). Si una variante tiene varias mallas, parentalas bajo un Empty llamado `leg_pants_dark` (los hijos se incluyen solos).

Cómo hacer los pantalones sin modelar ropa: un cilindro alrededor de la pantorrilla, un poco más ancho que la pierna, con 2-3 loops, escalado irregular (que caiga con algo de arruga — proportional editing **O** ayuda),

material Principled con textura de tela o color plano rugoso (Roughness ~0.9). No tiene que ser alta costura: tiene que ocultar el tobillo como lo hace un pantalón real.

Los 6 Empties de keypoints (nombres EXACTOS)

Agregá 6 Empties (`Shift+A` → `Empty` → `Plain Axes` , tamaño chico ~0.01) **pegados a la superficie del pie**, con estos nombres exactos (el script los busca; si no están, los aproxima por bounding box y pierde precisión):

Nombre	Dónde va (pie DERECHO)
<code>kp_heel</code>	Punto más trasero del talón, a la altura del suelo
<code>kp_toe</code>	Empeine sobre la 2ª cabeza metatarsal (donde "nace" el dedo índice del pie)
<code>kp_ankle_in</code>	Maléolo MEDIAL — el hueso del tobillo del lado INTERNO (mira al otro pie)
<code>kp_ankle_out</code>	Maléolo LATERAL — el hueso del tobillo del lado externo
<code>kp_ball</code>	Bola del pie (planta, bajo la 1ª cabeza metatarsal)
<code>kp_toe_tip</code>	Punta del dedo gordo

`ankle_in` = medial es **anatómico** (no "izquierda de la pantalla") — es lo que hace válido el espejado para generar el pie izquierdo.

Los keypoints se marcan aunque el zapato los tape. Casi todos (talón, bola, punta) quedan *dentro* del zapato: es correcto y buscado — el modelo tiene que aprender a inferir dónde está el pie **debajo** del zapato, que es de donde sale la pose para calzarle el zapato virtual. Poné cada Empty en su posición anatómica real, sin importar que el calzado lo cubra.

¿Qué es "parentar" los Empties al mesh del pie?

Parentar = emparentar: hacer que un objeto sea "hijo" de otro, de modo que cuando el padre se mueve, rota o escala, **los hijos lo siguen automáticamente** manteniendo su posición relativa. Es el equivalente 3D de pegar una etiqueta a una caja: movés la caja y la etiqueta va con ella.

Acá sirve para que los 6 Empties queden **pegados al pie**: si después movés o rotás el pie (o el script lo rota en cada render), los keypoints acompañan y siguen marcando el talón, la punta, etc. Si NO los parentás, quedan sueltos en el espacio y al rotar el pie los keypoints apuntan al vacío.

Cómo se hace (10 segundos): 1. Seleccioná los 6 Empties (clic en el primero, `Shift` + clic en los otros 5). 2. **Último**, con `Shift` + clic, seleccioná el **mesh del pie** — el padre es siempre el que se selecciona al final (queda con borde más claro). 3. `Ctrl+P` → elegí **Object (Keep Transform)**.

Para comprobarlo: en el Outliner los Empties ahora aparecen anidados debajo del pie, y si rotás el pie con `R` los Empties giran con él. El script respeta ese parenteo al importar el GLB.

Export

`File` → `Export` → `glTF 2.0 (.glb)` : formato **glb**, con **+Y up** (default del exporter), incluir los Empties (Include → seleccioná todo lo relevante o exportá con todo visible). Guardar como `training/models/foot.glb` .

2. Los zapatos: `training/models/shoes/*.glb`

- **≥10 modelos variados:** zapatilla urbana, running, bota, botín, sandalia, zapato formal, etc. Un solo modelo = el modelo de IA memoriza ese zapato. Fuentes: Sketchfab (filtrar **CC0/CC-BY**), BlenderKit free, Poly Haven. **Anotá la licencia de cada uno** en `training/models/ASSETS.md`.
 - **Alineación manual obligatoria** (esta es LA regla): importá `foot.glb` como referencia, importá el zapato, y movelo/rotalo/escalalo A MANO hasta que calce perfecto en el pie. Después: `Ctrl+A → All Transforms`, **borrá el pie de la escena**, y exportá SOLO el zapato como `shoes/<nombre>.glb`. Al importarlos juntos después, pie y zapato calzan sin ningún ajuste — los scripts jamás los tocan.
 - Zapatos de caña alta (botas): está bien que tapen parte de la pierna — la máscara los renderiza como clase "zapato" y a la pierna como "pierna", esa frontera es justo lo que queremos aprender.
 - Solo pie derecho (igual que el pie).
-

3. HDRIs y texturas de piso (descarga, 10 minutos)

- **HDRIs** → `training/hdri/` : en polyhaven.com/hdri (CC0), bajá 10-20 en 2K formato `.hdr` : interiores (living, cocina, oficina, estudio) y 3-4 exteriores.
- **Pisos** → `training/floor_textures/` : en polyhaven.com/textures (CC0) o fotos propias: madera clara/oscura, cerámica, alfombra, cemento, pasto — el JPG del diffuse/albedo alcanza (no hacen falta normales).

Si las carpetas están vacías el script usa luces y colores planos de fallback (funciona, pero transfiere peor al mundo real).

4. Validar tu GLB (automático — corré esto ANTES que nada)

Cada vez que exportes `foot.glb`, corré el validador. Revisa todo el contrato de esta guía (escala, orientación, origen, los 6 Empties con sus posiciones anatómicas, el lado medial, las 3 variantes `leg_*`, mallas cerradas, materiales) y te dice exactamente qué corregir:

```
cd training
"C:/Program Files/Blender Foundation/Blender 5.1/blender.exe" --background ^
--python validate_foot_glb.py -- --glb models/foot.glb
```

Salida tipo checklist con ✓/△/✗. Con `--render` genera además `validate_preview.jpg` con los keypoints proyectados sobre un render, para verificarlos a ojo. Exportá → validá → corregí → repetí hasta ver  **El GLB cumple el contrato**. Recién entonces pasá al preview de renders.

5. Verificar los renders (antes de renderizar nada en serio)

```
cd training
"C:/Program Files/Blender Foundation/Blender 5.1/blender.exe" --background ^
--python 0b_blender_render.py -- --foot models/foot.glb --shoe_dir models/shoes ^
--hdri_dir hdri --floor_dir floor_textures --out data_preview --preview
```

Abrí `training/data_preview/preview.html` y revisá el checklist: - [] El log dice **Pierna: ['leg_bare', 'leg_pants_dark', 'leg_pants_light']** (¡los 3 nombres!) - [] El log dice **[kp] usando Empties kp_* del GLB** (no "generados por bbox") - [] Pie y zapato perfectamente alineados en TODOS los renders - [] La máscara muestra 4 tonos (fondo/pierna/pie/zapato) y coincide con la imagen - [] Los keypoints marcan ✓ en los visibles - [] El sujeto ocupa una parte razonable del frame (~30-60%) - [] Pisos e iluminación DISTINTOS entre renders

Con ese checklist verde recién se pasa al piloto de 500 (GATE G4a del [ROADMAP](#)).

Errores comunes

- Exportar sin aplicar transforms → el pie aparece gigante/rotado. `Ctrl+A` siempre.
- Variantes de pierna en colecciones en vez de nombres de objeto → el GLB las pierde y el log dice "ningún objeto leg_*".
- `kp_ankle_in` puesto en el lado externo → el espejado produce pies imposibles; medial = interno.
- Mallas abiertas (huecos en el corte de la pierna) → se ve el interior hueco en ángulos bajos.
- Zapato alineado "a ojo" en el render final pero exportado sin aplicar la transform → desalineado al importar. Aplicar y verificar reimportando.

Documento generado desde `training/GUIA_ESCENA_BLENDER.md` — ante cualquier diferencia, el `.md` del repo es la fuente de verdad. Proyecto ar-shoes · ROADMAP.md Fases 3–4.